

课程编号 1503020001-01

题目类型 实验 2

得分	教师签名	批改日期
	冯禹洪	

深圳大学实验报告

课程名称： 系统编程

实验项目名称： 进程间通信

学院： 计算机与软件学院

专业： 计算机科学与技术（创新班）

指导教师： 冯禹洪

报告人： 李文俊 学号： 2023150001 班级： 高性能

实验时间： 2026. 5. 18

实验报告提交时间： 2026. 5. 18

教务处制

一、实验目标：

1. 掌握管道/命名管道的作用及其在编程中的作用
2. 掌握信号的作用及其在编程中的应用
3. 掌握客户端-服务器架构及其编程实现
4. 感受 C 库函数的实现与系统调用之间的关系

二、实验环境与工件

1. Taishan 服务器，openEuler 操作系统

三、实验内容与步骤

1. 客户端-服务器架构是常用的网络编程架构。使用我们现学的知识一个简单的客户端服务器可以描绘如图 1。

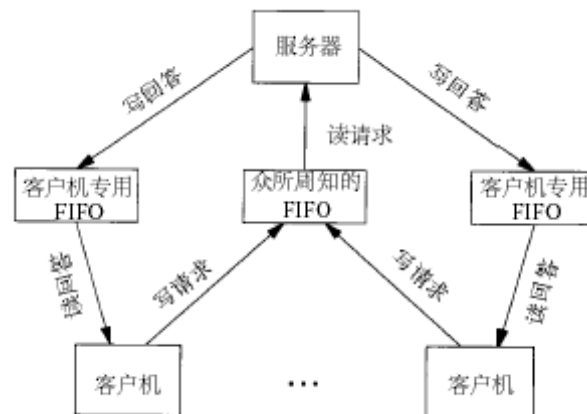


图 1. 客户端-服务器用 FIFO 进行通信

其中，服务器(Server)端的功能描述如下：

- 后台进程
- 从命名管道中读取客户端的请求
- 计算结果并将其写到客户端的命名管道

客户端(Client)的功能描述如下：

- 通过服务器命名管道发送请求到服务器
- 通过自己的命名管道读取服务器消息然后退出

注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。

2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。

用到的通信机制 FIFO 和信号处理。图 2 是客户端发送给服务器的请求的消息格式，图 3 是客户端的实现代码，图 4 是服务器端的实现代码。

```
/* clientinfo.h */

#ifndef _CLIENTINFO_H
#define _CLIENTINFO_H

typedef struct {
    char myfifo[50]; /* client's FIFO name */
    int leftarg;      /* left argument of calculation */
    int rightarg;     /* right argument of calculation */
    char op;          /* operation: + - "*" / */
} CLIENTINFO, *CLIENTINFOPTR;

#endif
```

图 2. 客户端发送给服务器的请求的消息格式

```
/* client.c */

#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/types.h>
#include <fcntl.h>
#include <signal.h>
#include "clientinfo.h"

#define FIFO_NAME "/tmp/server_fifo"
#define BUFF_SZ 100

char mypipename[BUFF_SZ];

void handler(int sig) { /* remove pipe if signaled */
    unlink(mypipename);
    exit(1);
}
```

- 注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。
2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。

```

int main(int argc, char *argv[]) {
    int res;
    int fifo_fd, my_fifo;
    int fd;
    CLIENTINFO info;
    char buffer[BUFF_SZ];

    /* handle some signals */
    signal(SIGKILL, handler);
    signal(SIGINT, handler);
    signal(SIGTERM, handler);

    /* check for proper command line */
    if (argc != 4) {
        printf("Usage: %s op1 operation op2\n", argv[0]);
        exit(1);
    }

    /* check if server fifo exists */
    if (access(FIFO_NAME, F_OK) == -1) {
        printf("Could not open FIFO %s\n", FIFO_NAME);
        exit(EXIT_FAILURE);
    }

    /* open server fifo for write */
    fifo_fd = open(FIFO_NAME, O_WRONLY);
    if (fifo_fd == -1) {
        printf("Could not open %s for write access\n",
            FIFO_NAME);
        exit(EXIT_FAILURE);
    }

    /* create my own FIFO */
    sprintf(mypipename, "/tmp/client%d_fifo", getpid());
    res = mkfifo(mypipename, 0777);
    if (res != 0) {
        printf("FIFO %s was not created\n", buffer);
        exit(EXIT_FAILURE);
    }

    /* open my own FIFO for reading */
    my_fifo = open(mypipename, O_RDONLY | O_NONBLOCK);
    if (my_fifo == -1) {
        printf("Could not open %s for read only access\n",
            FIFO_NAME);
        exit(EXIT_FAILURE);
    }
}

```

- 注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。
- 2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。

```

/* construct client info */
strcpy(info.myfifo, mypipename);
info.leftarg = atoi(argv[1]);
info.op = argv[2][0];
info.rightarg = atoi(argv[3]);

/* write client info to server fifo */
write(fifo_fd, &info, sizeof(CLIENTINFO));
close(fifo_fd);

```

```

/* get result from server */
memset(buffer, '\0', BUFF_SZ);
while (1) {
    res = read(my_fifo, buffer, BUFF_SZ);
    if (res > 0) {
        printf("Received from server: %s\n", buffer);
        break;
    }
}

printf("Client %d is terminating\n", getpid());

/* delete fifo from system */
close(my_fifo);
(void)unlink(mypipename);

exit(0);
}

```

图 3. client.c 的实现

```

/* calcserver.c */

#include <unistd.h>
#include <stdlib.h>
#include <stdio.h>
#include <string.h>
#include <sys/types.h>
#include <fcntl.h>
#include "clientinfo.h"

#define FIFO_NAME "/tmp/server_fifo"

void handler(int sig) {
    unlink(FIFO_NAME);
    exit(1);
}

```

- 注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。
- 2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。

```

int calc(CLIENTINFOPTR info) {
    switch(info->op) {
        case '+': return info->leftarg + info->rightarg;
        case '-': return info->leftarg - info->rightarg;
        case '*': return info->leftarg * info->rightarg;
        case '/': return info->leftarg / info->rightarg;
    }
    return 0;
}

int main() {
    int res;
    int i;
    int fifo_fd, fd1;
    CLIENTINFO info;
    char buffer[100];

    signal(SIGKILL, handler);
    signal(SIGINT, handler);
    signal(SIGTERM, handler);

    /* create FIFO, if necessary */
    if (access(FIFO_NAME, F_OK) == -1) {
        res = mkfifo(FIFO_NAME, 0777);
        if (res != 0) {
            printf("FIFO %s was not created\n", FIFO_NAME);
            exit(EXIT_FAILURE);
        }
    }

    /* open FIFO for reading */
    fifo_fd = open(FIFO_NAME, O_RDONLY);
    if (fifo_fd == -1) {
        printf("Could not open %s for read only access\n",
            FIFO_NAME);
        exit(EXIT_FAILURE);
    }

    printf("\nServer is rarin' to go!\n");

    while (1) {
        res = read(fifo_fd, &info, sizeof(CLIENTINFO));
        if (res != 0) {
            printf("Client arrived!!\n");
            sprintf(buffer, "The result is %d", calc(&info));
            fd1 = open(info.myfifo, O_WRONLY | O_NONBLOCK);
            write(fd1, buffer, strlen(buffer)+1);
            close(fd1);
        }
    }
    exit(0);
}

```

图 4. calcservice.c 的实现

- 注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。
- 2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。

请分析 Server 与 Client 的行为以及他们之间的交互，附上相应的运行结果。运行流程见图 5。另外，并用命令显示公共 FIFO 相关的信息并解释特征字段含义；（20 分）

```
$ ./calcserver &
[2] 3683
$ ./client 3 + 45

Server is rarin' to go!
Client arrived!!
Received from server: The result is 48
Client 3684 is terminating
$ ./client 3 "*" 2 &
[3] 3685
$ Client arrived!!
Received from server: The result is 6
Client 3685 is terminating
kill 3683
[3]- Done                               ./client 3 "*" 2
$
```

图 5 客户端-服务器的执行

2. 请在你的主目录如/home/szu 下创建目录，以你的姓名学号命名，例如 zhangsan123456, 在目录 zhangsan123456 下建子文件 chatapplication, 在 chatapplication 下建 data, 最后在 data 下分别建立 server_fifo, client_fifo, 请将公共命名管道设置为 /home/szu/zhangsan123456/chatapplication/data/server_fifo/chat_server_fifo 请将私有命名管道设置为 /home/szu/zhangsan123456/chatapplication/data/client_fifo/chat_client\$PID_fifo, 其中, \$PID 请用客户端进程 id 取代 请结合题干文件” GDB 的使用.ppt”和“用 GDB 调试多进程程序.jpg”, 使用 GDB 定位出错代码行和修正代码。请截图演示关键调试步骤、出错代码和修正代码段。（30 分）

3. 读 signals-hw2.c 中的每个代码段，推断其输出，并用运行程序验证结果：附上以下运行结果的截屏并分析。（20 分）

```
$gcc -o signals-hw2 signals-hw2.c
$./ signals-hw2-hw1 1
.....
$./ signals-hw2 4
```

4. 编写程序实现多个 Client 之间通过 Server 来传递消息从而实现 client 间的通信功能。要求如下

(1) 服务器创建 3 个众所周知的命名管道 FIFO_1, FIFO_2, FIFO_3, 分别接收用户的注册请求、登录请求和聊天请求，服务器等待任一管

注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。

2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。

道来的请求，收到时立即响应；不同用户不允许使用同一用户名注册，并设置密码。

- (2) 每个用户都建立一个以自己用户名为名字的命名管道，用户 A 发送给用户 B 的信息发送给服务器，然后服务器通过 B 的专用 FIFO 发送给用户 B。

请用文字描述你要实现的通信，相应的数据结构说明，并附上相应的代码及运行结果截图。（30 分）

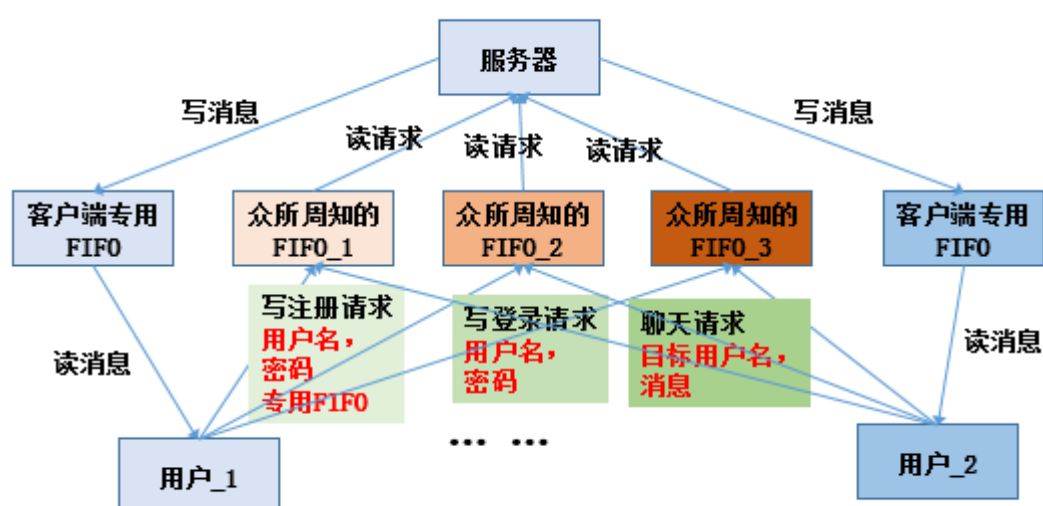


图 6 多用户聊天系统框架图

四、实验结果

1. GDB 调试原程序

1.1 原程序编译与调试准备

- (1) 首先进入实验目录/home/szu/liwenjun2023150001/chatapplication，将原始代码保存到 raw/文件夹下，方便后续与修改后的版本进行对比，对 raw/ 中的原始代码进行带调试信息编译
- (2) 原程序中使用了 mkfifo()，但源文件没有包含 sys/stat.h，在严格编译环境下可能出现 mkfifo 未声明的错误，因此加上需要的头文件后执行编译命令，-Wextra 显示更多的警告信息
 - a) `gcc -g -Wall -Wextra -o raw_calcserver raw/calcserver.c`
 - b) `gcc -g -Wall -Wextra -o raw_client raw/client.c`
- (3) 创建两个调试副本，避免破坏原始程序，执行相同的编译命令
 - a) `cp raw/calcserver.c gdb_calcserver.c`
 - b) `cp raw/client.c gdb_client.c`

注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。

2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。

```

[szu@euler-container-30001 chatapplication]$ gcc -g3 -Wall -Wextra -o gdb_calcserver gdb_calcserver.c && gdb -q ./gdb_calcserver
gdb_calcserver.c: In function 'handler':
gdb_calcserver.c:15:18: warning: unused parameter 'sig' [-Wunused-parameter]
   15 | void handler(int sig)
      |               ^
gdb_calcserver.c: In function 'main':
gdb_calcserver.c:40:9: warning: unused variable 'i' [-Wunused-variable]
   40 |     int i;
      |     ^

```

图 7 为 gdb_calcserver.c 的编译结果例

1.2 使用 GDB 定位公共 FIFO 路径错误

调试服务器程序：

- (1) 由于 FIFO_NAME 是宏定义，不是普通变量，不能直接用 print FIFO_NAME 查看；使用 -g3 编译后，可通过 **info macro FIFO_NAME** 查看宏内容
- (2) GDB 中可以观察到 FIFO_NAME = "/tmp/server_fifo"，这说明原程序的公共 FIFO 路径固定为 /tmp/server_fifo，根据实验要求，公共 FIFO 应放在实验目录的数据子目录下，所以这是一个路径错误。



```

szu@euler-container-30001:~/liwenjun2023150001/chatapplication
Reading symbols from ./gdb_calcserver...
(gdb) break main
Breakpoint 1 at 0x400a98: file gdb_calcserver.c, line 45.
(gdb) run
Starting program: /home/szu/liwenjun2023150001/chatapplication/gdb_calcserver
warning: Error disabling address space randomization: Function not implemented
Missing separate debuginfos, use: dnf debuginfo-install glibc-2.38-65.oe2403sp2
aarch64
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib64/libthread_db.so.1".

Breakpoint 1, main () at gdb_calcserver.c:45
45      signal(SIGKILL, handler);
(gdb) info macro FIFO_NAME
Defined at /home/szu/liwenjun2023150001/chatapplication/gdb_calcserver.c:13
#define FIFO_NAME "/tmp/server_fifo"
(gdb)

```

图 8 GDB 定位服务器公共 FIFO 路径错误截图。

1.3 使用 GDB 定位客户端私有 FIFO 路径错误

- (1) 先在一个终端后台运行服务器调试副本，保证 client 能顺利打开服务器 FIFO
- (2) 在另一个终端调试客户端：gdb ./gdb_client
 - a) (gdb) break main
 - b) (gdb) run 12 + 30
 - c) 执行到 client 把 getpid 作为模版的价值写入 mypipename 时，print mypipename

注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。
 2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。

(3) 可得到和服务端类似结果\$2 = "/tmp/client12345_fifo", 这说明原程序的客户端私有 FIFO 也创建在/tmp 目录下。根据实验要求, 客户端 FIFO 也应放在数据子目录下。

```
Reading symbols from ./gdb_client...
(gdb) break gdb_client.c:57
Breakpoint 1 at 0x400b5c: file gdb_client.c, line 57.
(gdb) run 12 + 30
Starting program: /home/szu/liwenjun2023150001/chatapplication/gdb_client 12 + 3
0
warning: Error disabling address space randomization: Function not implemented
Missing separate debuginfos, use: dnf debuginfo-install glibc-2.38-65.oe2403sp2.
aarch64
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib64/libthread_db.so.1".

Breakpoint 1, main (argc=4, argv=0xffffe1aafb58) at gdb_client.c:57
57      res = mkfifo(mypipename, 0777);
(gdb) print mypipename
$1 = "/tmp/client925982_fifo", '\000' <repeats 77 times>
(gdb) info macro FIFO_NAME
Defined at /home/szu/liwenjun2023150001/chatapplication/gdb_client.c:13
#define FIFO_NAME "/tmp/server_fifo"
(gdb)
```

图 9 GDB 查看客户端 mypipename 截图

1.4 用 GDB 定位 SIGKILL 处理无效

原服务器代码在 main() 中注册了 signal(SIGKILL, handler);, 在 GDB 中停在该行, 并直接调用同样的系统函数, 观察它的返回值:

```
gdb ./raw_calcserver_debug

(gdb) break raw/calcserver.c:44

(gdb) run

(gdb) list 42,46

(gdb) print (void (*)(int)) signal(SIGKILL, handler)

(gdb) print errno
```

观察到返回值为 0xffffffffffffff, 即 SIG_ERR; errno 为 22, 表示参数无效。这说明 SIGKILL 不能被捕获、阻塞或忽略, 给它注册处理函数没有效果。因此应删除 signal(SIGKILL, handler), 保留 SIGINT、SIGTERM 等可捕获信号即可。

注: 1、报告内的项目或内容设置, 可根据实际情况加以调整和补充。
2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。

```
szu@euler-container-30001:~/liwenjun2023150001/chatapplication
Reading symbols from ./raw_calcserver...
Breakpoint 1 at 0x400a98: file raw/calcserver.c, line 44.
Starting program: /home/szu/liwenjun2023150001/chatapplication/raw_calcserver
warning: Error disabling address space randomization: Function not implemented
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib64/libthread_db.so.1".

Breakpoint 1, main () at raw/calcserver.c:44
44      signal(SIGKILL, handler);
42      char buffer[100];
43
44      signal(SIGKILL, handler);
45      signal(SIGINT, handler);
46      signal(SIGTERM, handler);
$1 = (void (*)(int)) 0xffffffffffffffff
$2 = 22
Missing separate debuginfos, use: dnf debuginfo-install glibc-2.38-65.oe2403sp2.
aarch64
A debugging session is active.

        Inferior 1 [process 927381] will be killed.

Quit anyway? (y or n) ☐
```

图 10 sigkill 非法返回值

1.5 用 GDB 定位客户端错误分支打印未初始化 buffer

原客户端在创建私有 FIFO 失败时执行: `printf("FIFO %s was not created\n", buffer);`

但此时 `buffer` 尚未被赋值, 为了稳定进入失败分支, GDB 停在 `mkfifo(mypipename, 0777)` 前, 先手动调用一次 `mkfifo()` 创建同名 FIFO, 再执行原代码中的 `mkfifo()`, 人为制造系统调用失败, 然后观察返回值和 `errno`:

```
gdb ./raw_client_debug
(gdb) break raw/client.c:56
(gdb) run 12 + 30
(gdb) print mypipename
(gdb) call (int)mkfifo(mypipename, 0777)
(gdb) next
(gdb) print res
(gdb) print errno
(gdb) list 55,59
(gdb) print buffer
```

- 注: 1、报告内的项目或内容设置, 可根据实际情况加以调整和补充。
2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。


```
(gdb) set var info.rightarg = 0
```

```
(gdb) set var info.op = '/'
```

```
(gdb) print info
```

```
(gdb) call calc(&info)
```

```
(gdb) list 29,32
```

GDB 停在 `return info->leftarg / info->rightarg;`, 并且变量显示 `rightarg = 0`, 这说明原程序在

除数为 0 时仍直接执行除法, 没有错误处理, 从而触发运行时异常。修正时应在除法前判断

`info->rightarg == 0`, 并返回 **Error: division by zero.**

```
(gdb) set var info.leftarg = 81
(gdb) set var info.rightarg = 0
(gdb) set var info.op = '/'
(gdb) print info
$1 = {
  myfifo = "\000\000\353\264\377\377", '\000' <repeats 12 times>, "\353\264\377\377\000\000\361\377", '\000' <repeats 14 times>, "\377\377\377\377\000\000\000\000\000", <incomplete sequence \332>, leftarg = 81, rightarg = 0, op = 47 '/' }
(gdb) call calc(&info)

Breakpoint 1, calc (info=0xffffd54cd900) at raw/calccserver.c:31
31      return info->leftarg / info->rightarg;
The program being debugged stopped while in a function called from GDB.
Evaluation of the expression containing the function
(calc) will be abandoned.
When the function is done executing, GDB will silently stop.
(gdb) list 29,32
29      return info->leftarg * info->rightarg;
30      case '/':
31      return info->leftarg / info->rightarg;
32      }
(gdb) next
34      }
(gdb) print info->leftarg
$2 = 81
(gdb) print info->rightarg
$3 = 0
(gdb) □
```

图 12 定位除零错误

1.7 GDB 定位服务器只读打开 FIFO 后读到 EOF

原服务器代码使用 `fifo_fd = open(FIFO_NAME, O_RDONLY);` 客户端发送请求并关闭写端后, 服务器下一次 `read()` 可能返回 0, GDB 中在服务器 `read()` 行设置断点, 另一个进程运行客户端:

```
gdb ./raw_calccserver_debug
```

```
(gdb) break raw/calccserver.c:72
```

```
(gdb) run
```

注: 1、报告内的项目或内容设置, 可根据实际情况加以调整和补充。

2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。

客户端执行一次 `./raw_client_debug 12 + 30` 后，服务器第一次 `read()` 返回完整结构体大小，第二次回到同一行再执行 `next`:

```
(gdb) next
```

```
(gdb) print res
```

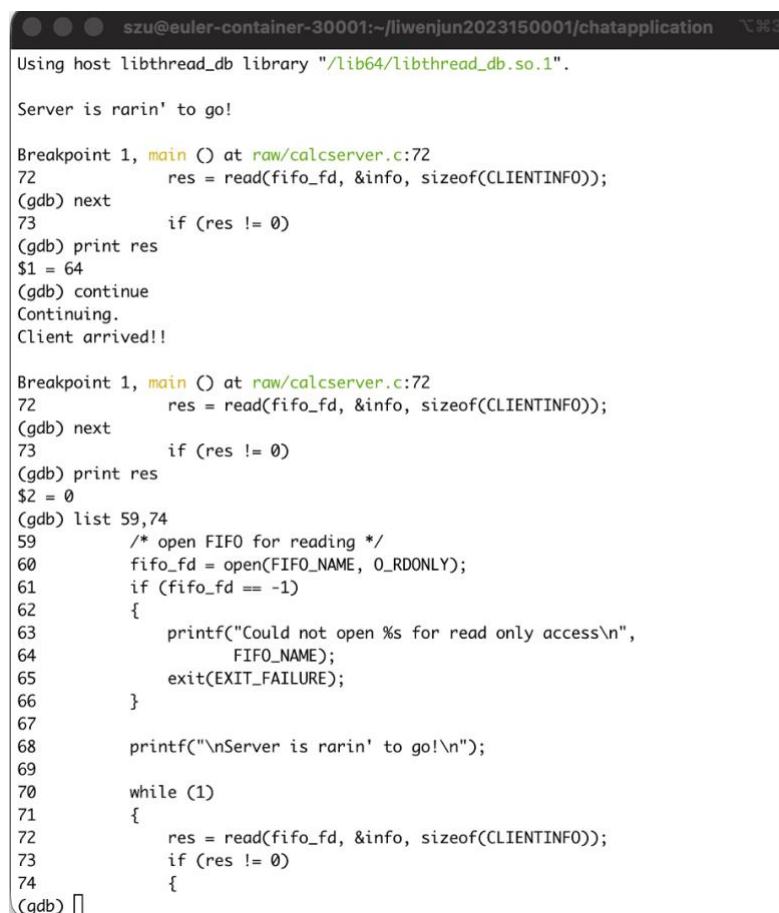
```
(gdb) continue
```

```
(gdb) next
```

```
(gdb) print res
```

```
(gdb) list 59,74
```

GDB 中可见第一次 `res = 64`，说明读到了一个完整请求；客户端关闭后，第二次 `res = 0`，说明读到了 EOF。原代码只判断 `if (res != 0)`，没有专门处理 EOF，会导致服务器在循环中无意义空转，修正时将服务器公共 FIFO 改为 `O_RDWR` 打开，可以避免无写端时反复读到 EOF。



```
szu@euler-container-30001:~/liwenjun2023150001/chatapplication %3
Using host libthread_db library "/lib64/libthread_db.so.1".

Server is rarin' to go!

Breakpoint 1, main () at raw/calcservice.c:72
72      res = read(fifo_fd, &info, sizeof(CLIENTINFO));
(gdb) next
73      if (res != 0)
(gdb) print res
$1 = 64
(gdb) continue
Continuing.
Client arrived!!

Breakpoint 1, main () at raw/calcservice.c:72
72      res = read(fifo_fd, &info, sizeof(CLIENTINFO));
(gdb) next
73      if (res != 0)
(gdb) print res
$2 = 0
(gdb) list 59,74
59      /* open FIFO for reading */
60      fifo_fd = open(FIFO_NAME, O_RDONLY);
61      if (fifo_fd == -1)
62      {
63          printf("Could not open %s for read only access\n",
64                FIFO_NAME);
65          exit(EXIT_FAILURE);
66      }
67
68      printf("\nServer is rarin' to go!\n");
69
70      while (1)
71      {
72          res = read(fifo_fd, &info, sizeof(CLIENTINFO));
73          if (res != 0)
74          {
(gdb) █
```

图 13 服务器只读打开 FIFO 后读到 EOF 错误

2. FIFO 客户端-服务器修正与运行

2.1 路径修正

- 注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。
2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。

修正后的程序仍采用“公共 FIFO + 私有 FIFO”的结构，服务器创建一个公共 FIFO 接收所有客户端请求，客户端创建自己的私有 FIFO 接收服务器返回结果。

公共 FIFO 路径为：chatapplication/data/server_fifo/chat_server_fifo

客户端私有 FIFO 路径格式为：chatapplication/data/client_fifo/chat_client<PID>_fifo

在 clientinfo.h 集中定义实验目录和 FIFO 路径，方便统一维护，同时扩大 CLIENTINFO.myfifo 的长度，满足绝对路径长度，避免字符串截断风险。

```
/* 应用数据与FIFO端点的基础路径。 */
#define CALC_USER_HOME "/home/szu/liwenjun2023150001" /* 用户主目录 */
#define CALC_APP_DIR CALC_USER_HOME "/chatapplication" /* 应用根目录 */
#define CALC_DATA_DIR CALC_APP_DIR "/data" /* 数据目录 */
#define CALC_SERVER_FIFO_DIR CALC_DATA_DIR "/server_fifo" /* 服务器FIFO目录 */
#define CALC_CLIENT_FIFO_DIR CALC_DATA_DIR "/client_fifo" /* 客户端FIFO目录 */

/* FIFO名称：服务器公共FIFO与每个客户端FIFO模板。 */
#define FIFO_NAME CALC_SERVER_FIFO_DIR "/chat_server_fifo" /* 服务器公共FIFO路径 */
#define CLIENT_FIFO_TEMPLATE CALC_CLIENT_FIFO_DIR "/chat_client%d_fifo" /* 客户端FIFO模板 */

/* FIFO路径和服务端回复的缓冲区大小。 */
#define CLIENT_FIFO_NAME_LEN 256 /* 客户端FIFO路径缓冲区长度 */
#define SERVER_REPLY_LEN 128 /* 服务器回复缓冲区长度 */

typedef struct
{
    char myfifo[CLIENT_FIFO_NAME_LEN]; /* 客户端回复FIFO路径 */
}
```

图 14 路径定义代码

```
static char mypipename[CLIENT_FIFO_NAME_LEN]; /* 客户端FIFO路径缓存 */
snprintf(mypipename, sizeof(mypipename), CLIENT_FIFO_TEMPLATE, getpid());
my_fifo = open(mypipename, O_RDONLY | O_NONBLOCK); /* 非阻塞打开客户端FIFO */
```

图 15 客户端使用正确路径代码

2.2 自动创建实验目录

若 data/server_fifo 或 data/client_fifo 不存在，mkfifo() 会失败。自动创建目录可以减少手工准备步骤，通过增加 ensure_dir()、prepare_fifo() 或 prepare_dirs()，启动时依次创建用户目录、应用目录、数据目录、服务器 FIFO 目录和客户端 FIFO 目录

注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。

2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。

```
static int prepare_dirs(void)
{
    if (ensure_dir(CALC_USER_HOME) == -1 || /* 用户主目录 */
        ensure_dir(CALC_APP_DIR) == -1 || /* 应用目录 */
        ensure_dir(CALC_DATA_DIR) == -1 || /* 数据目录 */
        ensure_dir(CALC_SERVER_FIFO_DIR) == -1 || /* 服务器FIFO目录 */
        ensure_dir(CALC_CLIENT_FIFO_DIR) == -1) /* 客户端FIFO目录 */
        return -1; /* 任意失败则返回 */

    return 0; /* 目录准备成功 */
}
```

图 16 自动创建实验目录代码

2.3 去掉无效的 SIGKILL 处理

calcserver.c、client.c 里 SIGKILL 不能被捕获、阻塞或忽略，给它注册处理函数没有效果，因此保留 SIGINT、SIGTERM、SIGHUP 的清理处理，删除 signal(SIGKILL, handler)

```
signal(SIGINT, handler); /* Ctrl+C */
signal(SIGTERM, handler); /* 终止信号 */
signal(SIGHUP, handler); /* 终端挂起 */
```

图 17 去掉无效的 SIGKILL 代码

2.4 增加资源清理函数

calcserver.c、client.c 中程序异常退出或按 Ctrl+C 结束时，应关闭文件描述符并删除 FIFO，避免遗留无用管道文件，服务器使用 cleanup() 关闭公共 FIFO 并删除服务器 FIFO；客户端使用 cleanup() 关闭私有 FIFO 并删除自己的 FIFO

```
static void handler(int sig)
{
    (void) sig; /* 避免未使用参数警告 */
    cleanup(EXIT_FAILURE); /* 异常退出时清理 */
}

static void cleanup(int status)
{
    if (my_fifo != -1) /* 描述符有效则关闭 */
        close(my_fifo); /* 关闭客户端FIFO */
    if (mypipename[0] != '\0') /* FIFO路径已生成 */
        unlink(mypipename); /* 删除客户端FIFO文件 */
    exit(status); /* 退出进程 */
}
```

图 18 客户端资源清理函数代码

- 注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。
2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。

```

static void handler(int sig)
{
    (void)sig;          /* 避免未使用参数警告 */
    /* cleanup(EXIT_FAILURE); /* 异常退出时清理 */
}

static void cleanup(int status)
{
    if (fifo_fd != -1) /* 描述符有效则关闭 */
        close(fifo_fd); /* 关闭服务器FIFO */
    unlink(FIFO_NAME); /* 删除公共FIFO */
    exit(status);       /* 退出进程 */
}

```

图 19 服务器资源清理函数代码

2.5 服务器以读写方式打开公共 FIFO

原程序用 O_RDONLY 打开公共 FIFO，当没有客户端写入端时可能读到 EOF，不利于长期持续运

行：`fifo_fd = open(FIFO_NAME, O_RDWR);`

2.6 增加除零和非法运算符处理

服务器原程序直接执行除法，若右操作数为 0 会触发错误；非法运算符也没有明确提示：

```

case '/':
    if (info->rightarg == 0) /* 除零保护 */
    {
        snprintf(buffer, size, "Error: division by zero"); /* 输出错误 */
        return; /* 提前返回 */
    }
    result = info->leftarg / info->rightarg; /* 除法 */
    break; /* 结束分支 */
default:
    snprintf(buffer, size, "Error: unsupported operator '%c'", info->op); /* 不支持运算符 */
    return;

```

图 20 除零和非法运算符处理代码

2.7 修正客户端错误输出

原客户端程序在 `mkfifo()` 失败时打印未初始化的 `buffer`，错误信息不可靠，修改为将错误输出改为 `perror(mypipename)`，直接显示创建失败的 FIFO 路径和系统错误原因

```

if (mkfifo(mypipename, 0600) == -1)
{
    perror(mypipename); /* 输出创建失败原因 */
    exit(EXIT_FAILURE); /* 退出 */
}

```

图 21 客户端错误输出代码

2.8 客户端增加参数和运算符校验

注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。

2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。

原程序只检查参数个数，未检查运算符是否合法，增加 parse_args()，要求运算符只能是 +、-、*、/

```
static int parse_args(int argc, char *argv[], CLIENTINFOPTR info)
{
    if (argc != 4) /* 参数数量校验 */
    {
        fprintf(stderr, "Usage: %s op1 operation op2\n", argv[0]); /* 用法提示 */
        return -1; /* 参数数量错误 */
    }

    if (strlen(argv[2]) != 1 || strchr("+-*/", argv[2][0]) == NULL) /* 运算符校验 */
    {
        fprintf(stderr, "Operation must be one of: + - '*' /\n"); /* 运算符提示 */
        return -1; /* 运算符非法 */
    }

    info->leftarg = atoi(argv[1]); /* 左操作数 */
    info->op = argv[2][0]; /* 运算符 */
    info->rightarg = atoi(argv[3]); /* 右操作数 */
    return 0; /* 解析成功 */
}
```

图 22 参数和运算符校验代码

2.9 客户端轮询读取时处理可重试错误

原客户端程序以 O_NONBLOCK 打开私有 FIFO，暂时没有数据时 read() 可能返回 EAGAIN 或 EWOULDBLOCK，这不是致命错误，因此读取循环中忽略 EAGAIN、EWOULDBLOCK、EINTR，并用 usleep(10000) 降低空转消耗。

```
while (1) /* 等待回复 */
{
    nread = read(my_fifo, buffer, sizeof(buffer)); /* 尝试读取回复 */
    if (nread > 0) /* 收到回复 */
    {
        printf("Received from server: %s\n", buffer); /* 打印结果 */
        break; /* 结束等待 */
    }

    if (nread == -1 && errno != EAGAIN && errno != EWOULDBLOCK && errno != EINTR) /* 非可重试错误 */
    {
        perror("read"); /* 读取失败 */
        cleanup(EXIT_FAILURE); /* 清理并退出 */
    }

    usleep(10000); /* 10ms 轮询间隔 */
}
```

图 23 轮询读取时处理可重试代码

2.10 运行验证

服务器终端：./calcservice，输出：

- 注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。
2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。

```
[szu@euler-container-30001 chatapplication]$ ./calcserver
Server is ready on /home/szu/liwenjun2023150001/chatapplication/data/server_fifo
/chat_server_fifo
```

图 24 服务器运行初始输出

客户端终端：

```
./client 12 + 30

./client 50 - 8

./client 7 '*' 6

./client 81 / 9

./client 81 / 0
```

```
• [szu@euler-container-30001 chatapplication]$ ./client 12 + 30
Received from server: The result is 42
Client 928433 is terminating
• [szu@euler-container-30001 chatapplication]$ ./client 50 - 8
Received from server: The result is 42
Client 928458 is terminating
• [szu@euler-container-30001 chatapplication]$ ./client 7 '*' 6
Received from server: The result is 42
Client 928480 is terminating
• [szu@euler-container-30001 chatapplication]$ ./client 81 / 9
Received from server: The result is 9
Client 928499 is terminating
• [szu@euler-container-30001 chatapplication]$ ./client 81 / 0
Received from server: Error: division by zero
Client 928515 is terminating
```

图 25 客户端输出

服务器端可看到类似输出，证明服务器不会读到 EOF，能够长期稳定运行：

```
[szu@euler-container-30001 chatapplication]$ ./calcserver
Server is ready on /home/szu/liwenjun2023150001/chatapplication/data/server_fifo
/chat_server_fifo
Client arrived: /home/szu/liwenjun2023150001/chatapplication/data/client_fifo/ch
at_client928433_fifo, expression: 12 + 30
Client arrived: /home/szu/liwenjun2023150001/chatapplication/data/client_fifo/ch
at_client928458_fifo, expression: 50 - 8
Client arrived: /home/szu/liwenjun2023150001/chatapplication/data/client_fifo/ch
at_client928480_fifo, expression: 7 * 6
Client arrived: /home/szu/liwenjun2023150001/chatapplication/data/client_fifo/ch
at_client928499_fifo, expression: 81 / 9
Client arrived: /home/szu/liwenjun2023150001/chatapplication/data/client_fifo/ch
at_client928515_fifo, expression: 81 / 0
```

图 26 服务端输出

3. Server 与 Client 行为及交互分析

3.1 Server 行为分析

修正后的 `calcserver` 是一个长期运行的服务器进程。

启动后，它首先保证实验目录和 FIFO 目录存在，然后创建公共 FIFO：

- 注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。
2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。

/home/szu/liwenjun2023150001/chatapplication/data/server_fifo/chat_server_fifo

服务器使用 `open(FIFO_NAME, O_RDWR)` 打开公共 FIFO，并进入**无限循环**。每次循环中，服务器从公共 FIFO 读取一个 `CLIENTINFO` 请求包，该结构体中包含客户端私有 FIFO 路径、左操作数、右操作数和运算符。服务器读取到完整请求后，根据运算符完成计算，再用 `open(info.myfifo, O_WRONLY | O_NONBLOCK)` 打开对应客户端的私有 FIFO，把结果字符串写回客户端。

服务器端在交互中承担三个职责：

- (1) 提供固定的公共 FIFO 入口
- (2) 解析和计算客户端请求
- (3) 根据请求包中的 `myfifo` 字段把结果定向返回给发起请求的客户端

3.2 Client 行为分析

修正后的 client 是**一次请求、一次响应**的客户端进程。

客户端启动后先**解析命令行参数**，例如 `./client 3 + 45`，随后客户端根据自己的进程 id 生成私有 FIFO 路径：

/home/szu/liwenjun2023150001/chatapplication/data/client_fifo/chat_client<PID>_fifo

- (1) 客户端创建并打开该私有 FIFO，用于等待服务器返回结果
- (2) 然后客户端构造 `CLIENTINFO` 请求包，将自己的私有 FIFO 路径、操作数和运算符写入服务器公共 FIFO
- (3) 写入完成后，客户端关闭公共 FIFO 写端，转而从私有 FIFO 中等待服务器回复
- (4) 收到结果后，客户端打印结果、关闭私有 FIFO，并删除该私有 FIFO 文件

客户端端在交互中承担三个职责：

- (1) 为本次请求创建唯一的返回通道
- (2) 把请求写入服务器公共 FIFO
- (3) 从自己的私有 FIFO 接收服务器返回值并清理资源

3.3 Server 与 Client 交互流程

本实验按题目流程图的运行流程进行测试。先编译程序：

```
gcc -g -Wall -Wextra -o calcserver calcserver.c
```

```
gcc -g -Wall -Wextra -o client client.c
```

然后后台启动服务器：`./calcserver &`，终端输出服务器后台进程号，并显示服务器已经创建并打开

公共 FIFO：

注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。

2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。

```
[szu@euler-container-30001 chatapplication]$ ./calcserver
Server is ready on /home/szu/liwenjun2023150001/chatapplication/data/server_fifo
/chat_server_fifo
```

图 27 服务器输出结果

运行第一个客户端: ./client 3 + 45, 服务器收到请求并打印客户端私有 FIFO 路径和表达式, 客户端收到服务器返回结果:

```
Client arrived: /home/szu/liwenjun2023150001/chatapplication/data/client_fifo/ch
at_client928548_fifo, expression: 3 + 45
Received from server: The result is 48
Client 928548 is terminating
```

图 28 服务器处理客服端 1 请求并输出结果

运行第二个客户端 ./client 3 '*' 2 &, 并参考流程图将其放到后台执行, 最后结束服务器 kill

<server_pid>, 运行结果如下:

```
[2] 928549
Client arrived: /home/szu/liwenjun2023150001/chatapplication/data/client_fifo/ch
at_client928549_fifo, expression: 3 * 2
Received from server: The result is 6
Client 928549 is terminating
[2]+ Done ./client 3 \* 2
```

图 29 服务器处理客服端 2 请求并输出结果

3.4 公共 FIFO 信息及字段解释

在服务器运行期间, 用如下命令查看公共 FIFO 信息:

```
ls -l /home/szu/liwenjun2023150001/chatapplication/data/server_fifo/chat_server_fifo
```

```
stat /home/szu/liwenjun2023150001/chatapplication/data/server_fifo/chat_server_fifo
```

实际输出如下:

```
===== public FIFO information =====
prw-r--r--. 1 szu szu 0 May 16 15:09 /home/szu/liwenjun2023150001/chatapplicatio
n/data/server_fifo/chat_server_fifo
File: /home/szu/liwenjun2023150001/chatapplication/data/server_fifo/chat_serve
r_fifo
Size: 0          Blocks: 0          IO Block: 4096   fifo
Device: 0,70     Inode: 9773128    Links: 1
Access: (0644/prw-r--r--)  Uid: ( 1000/   szu)   Gid: ( 1000/   szu)
Access: 2026-05-16 15:09:10.284000000 +0000
Modify: 2026-05-16 15:09:10.284000000 +0000
Change: 2026-05-16 15:09:10.284000000 +0000
Birth: -
```

图 30 公共 FIFO 信息

字段含义如下:

- (1) p: ls -l 第一列第一个字符为 p, 表示该文件类型是 FIFO 命名管道, 而不是普通文件或目录

注: 1、报告内的项目或内容设置, 可根据实际情况加以调整和补充。

2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。

- (2) **rw-r--r--**: 表示 FIFO 文件节点的权限。所有者可读写，同组用户和其他用户可读。实际通信还需要进程以合适方式调用 open/read/write
- (3) **1 或 Links: 1**: 表示该 FIFO 目录项的**硬链接数**，通常为 1。
- (4) **szu szu 或 Uid/Gid**: 表示 FIFO 的**所有者和所属用户组**。
- (5) **Size: 0**: FIFO 不像普通文件那样把数据保存在磁盘文件内容中，消息在**内核缓冲区**中临时传递，因此大小通常显示为 0。
- (6) **Blocks: 0**: 表示该 FIFO **没有占用普通文件数据块**。
- (7) **IO Block: 4096**: 表示文件系统建议的 **I/O 块大小**，不代表 FIFO 中实际保存了 4096 字节数据。
- (8) **fifo**: stat 输出的**文件类型**，明确说明该路径是命名管道。
- (9) **Device 和 Inode**: 分别表示所在设备号和 inode 编号，用于在文件系统中唯一标识该目录项。
- (10) **Access/Modify/Change**: 分别表示最近访问时间、内容修改时间和状态改变时间。
FIFO 创建、权限或元数据变化会影响这些时间字段。
- (11) **Birth: -**: 表示当前文件系统或环境没有提供创建时间

4. signals-hw2.c 信号实验

4.1 borkfork

核心原理:

borkfork() 中宏 #define bork fork 使程序连续执行三次 fork(), 每一次 fork() 都会让父子进程继续执行后续代码, 因此最终共有 $2^3 = 8$ 个进程执行 printf("borked\n"), 预期输出 8 行 borked, 顺序由进程调度决定, 如果父进程较早结束, shell 可能会先打印命令提示符; 但其他子进程仍然可能继续向同一个终端输出 borked。因此可能看到提示符和 borked 混在一起

运行结果

```
● [szu@euler-container-30001 chatapplication]$ ./signals-hw2 borkfork
borked
borked
borked
borked
○ [szu@euler-container-30001 chatapplication]$ borked
borked
borked
borked
```

图 31 borkfork 运行结果

- 注: 1、报告内的项目或内容设置, 可根据实际情况加以调整和补充。
2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。

4.2 fork1

核心原理:

fork1() 循环创建 5 个子进程, 创建后让每个子进程进入死循环, 父进程随后对每个子进程发送 SIGINT。由于没有自定义 SIGINT 处理函数, 子进程按默认动作异常终止, 不属于正常退出 WIFEXITED, 当父进程使用 wait() 回收子进程时, 通过 status 状态值可以判断子进程属于异常退出, 预期所以子进程 pid 都会输出异常退出的字符串。

运行结果

```
[szu@euler-container-30001 chatapplication]$ ./signals-hw2 1
Killing process 931955
Killing process 931956
Killing process 931957
Killing process 931958
Killing process 931959
Child 931955 terminated abnormally
Child 931956 terminated abnormally
Child 931958 terminated abnormally
Child 931959 terminated abnormally
Child 931957 terminated abnormally
[szu@euler-container-30001 chatapplication]$
```

图 32 fork1 运行结果

4.3 fork2

核心原理:

fork2() 在 fork1() 的基础上注册了 SIGINT 处理函数 int_handler, 子进程会继承该信号处理设置, 因此收到 SIGINT 后进入 int_handler(), 打印信号编号并调用 exit(0) 正常退出, 所以预计父进程回收的状态值会表现为正常退出, WIFEXITED = True。

运行结果

```
[szu@euler-container-30001 chatapplication]$ ./signals-hw2 2
Killing process 932041
Killing process 932042
Killing process 932043
Killing process 932044
Killing process 932045
Process 932042 received signal 2
Child 932042 terminated with exit status 0
Process 932041 received signal 2
Child 932041 terminated with exit status 0
Process 932045 received signal 2
Child 932045 terminated with exit status 0
Process 932043 received signal 2
Child 932043 terminated with exit status 0
Process 932044 received signal 2
Child 932044 terminated with exit status 0
[szu@euler-container-30001 chatapplication]$
```

图 33 fork2 运行结果

4.4 fork3

核心原理:

- 注: 1、报告内的项目或内容设置, 可根据实际情况加以调整和补充。
2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。

fork3() 使用 SIGCHLD 处理函数回收子进程, SIGCHLD 是子进程状态发生变化时, 内核发给父进程的信号, 但处理函数每次只调用一次 wait()。5 个子进程因为 sleep 了 1s 后几乎同时退出。

由于标准信号不排队, 父进程还没来得及处理第一个 SIGCHLD, 后面又来了几个 SIGCHLD, 它们可能只合并成一个 pending SIGCHLD, 因此可能只回收部分子进程, 导致 **ccount** 不能降到 0, 程序停在 pause(), 此时成功没有回收的进程会变成僵尸进程。

运行结果:

```
[szu@euler-container-30001 chatapplication]$ ./signals-hw2 3
Received SIGCHLD signal 17 for process 932347
Received SIGCHLD signal 17 for process 932348
█
```

图 34 fork3 运行结果

验证结果:

通过 ps -o ppid --pid 932349 找到父进程 pid, 再通过 ps -o pid,ppid,stat,cmd --ppid 932346 查询没回收的子进程状态

```
● [szu@euler-container-30001 chatapplication]$ ps -o ppid --pid 932349
  PPID
 932346
● [szu@euler-container-30001 chatapplication]$ ps -o pid,ppid,stat,cmd --ppid 932346
  PID   PPID  STAT  CMD
 932349 932346 Z+    [signals-hw2] <defunct>
 932350 932346 Z+    [signals-hw2] <defunct>
 932351 932346 Z+    [signals-hw2] <defunct>
○ [szu@euler-container-30001 chatapplication]$ █
```

图 35 fork3 运行结果

正确写法:

handler 里不要只 wait() 一次, 而是循环回收所有已经退出的子进程

```
void child_handler(int sig)
{
    int child_status;
    pid_t pid;

    while ((pid = waitpid(-1, &child_status, WNOHANG)) > 0) {
        ccount--;
        printf("Received SIGCHLD signal %d for process %d\n", sig, pid);
        fflush(stdout);
    }
}
```

图 36 fork3 正确写法代码

正确运行结果:

- 注: 1、报告内的项目或内容设置, 可根据实际情况加以调整和补充。
2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。

```

[shu@euler-container-30001 chatapplication]$ gcc -o signals-hw2 signals-hw2.c
[shu@euler-container-30001 chatapplication]$ ./signals-hw2 3
Received SIGCHLD signal 17 for process 932541
Received SIGCHLD signal 17 for process 932542
Received SIGCHLD signal 17 for process 932543
Received SIGCHLD signal 17 for process 932544
Received SIGCHLD signal 17 for process 932540
[shu@euler-container-30001 chatapplication]$

```

图 37 fork3 正确运行结果

4.5 fork4

核心原理:

fork4() 使用 `child_handler2()`，正好对应了做 fork3() 实验时的代码修正，在信号处理函数中通过循环 `wait()` 回收所有已经终止的子进程。因此即使多个 SIGCHLD 合并，处理函数仍能一次回收多个子进程，使 `ccount` 最终降为 0。运行结果应与 fork3 的修正结果相同。

运行结果

```

[shu@euler-container-30001 chatapplication]$ ./signals-hw2 4
Received signal 17 from process 933032
Received signal 17 from process 933033
Received signal 17 from process 933034
Received signal 17 from process 933035
Received signal 17 from process 933036
[shu@euler-container-30001 chatapplication]$

```

图 38 fork4 运行结果

5. 多用户聊天系统

5.1 通信模型

多用户聊天系统采用一个服务器和多个客户端进程。服务器创建 3 个公共 FIFO，其中 FIFO_1 接收注册请求，FIFO_2 接收登录请求，FIFO_3 接收聊天请求

1. `/home/szu/liwenjun2023150001/chatapplication/data/server_fifo/FIFO_1`
2. `/home/szu/liwenjun2023150001/chatapplication/data/server_fifo/FIFO_2`
3. `/home/szu/liwenjun2023150001/chatapplication/data/server_fifo/FIFO_3`

每个客户端以用户名创建自己的私有 FIFO，例如：

- `/home/szu/liwenjun2023150001/chatapplication/data/client_fifo/alice`
- `/home/szu/liwenjun2023150001/chatapplication/data/client_fifo/bob`

服务器使用 `select()` 同时监听 3 个公共 FIFO：收到注册请求时检查用户名是否重复；收到登录请求时检查用户名和密码；收到聊天请求时查找接收方，并将消息写入接收方的私有 FIFO

5.2 数据结构说明

注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。

2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。

聊天系统把公共路径、长度限制和通信结构体统一放在 chat_common.h 中，这样服务器和客户端编译时使用同一套协议定义，避免双方对 FIFO 路径或结构体大小理解不一致。

路径宏定义如下：

```
#define CHAT_USER_HOME "/home/szu/liwenjun2023150001"
#define CHAT_APP_DIR CHAT_USER_HOME "/chatapplication"
#define CHAT_DATA_DIR CHAT_APP_DIR "/data"
#define CHAT_SERVER_FIFO_DIR CHAT_DATA_DIR "/server_fifo"
#define CHAT_CLIENT_FIFO_DIR CHAT_DATA_DIR "/client_fifo"

#define CHAT_FIFO_REGISTER CHAT_SERVER_FIFO_DIR "/FIFO_1"
#define CHAT_FIFO_LOGIN CHAT_SERVER_FIFO_DIR "/FIFO_2"
#define CHAT_FIFO_MESSAGE CHAT_SERVER_FIFO_DIR "/FIFO_3"
```

图 39 路径宏定义

CHAT_FIFO_REGISTER、CHAT_FIFO_LOGIN、CHAT_FIFO_MESSAGE 分别对应注册、登录和聊天消息 3 个公共 FIFO。

客户端不共用返回 FIFO，而是在 client_fifo 目录下按用户名创建私有 FIFO，例如 alice 客户端对应 /home/szu/liwenjun2023150001/chatapplication/data/client_fifo/alice，这样服务器可以根据用户表中保存的 FIFO 路径，把回复或聊天消息定向写回某一个客户端。

注册和登录请求共用结构体 ChatAuthRequest：username 保存用户名，password 保存密码，fifo 保存该客户端私有 FIFO 路径。

注册时服务器根据 username 判断是否重复，并保存 password 和 fifo；登录时服务器根据 username 找到用户，再比较 password，登录成功后刷新该用户的 fifo 并把在线状态置为 1。

注册和登录能共用该结构体，是因为这两种请求都需要身份信息和回包地址。

```
typedef struct
{
    char username[CHAT_NAME_LEN];
    char password[CHAT_PASSWORD_LEN];
    char fifo[CHAT_FIFO_PATH_LEN];
} ChatAuthRequest;
```

图 40 注册和登录请求共用结构体

聊天请求结构体 ChatSendRequest：from 表示发送者用户名，to 表示接收者用户名，text 表示聊天内容。服务器收到该结构体后，先在用户表中检查 from 是否存在并在线，再检查 to 是否存在并在线，最后把消息封装成服务器到客户端的统一数据包，写入接收方私有 FIFO。

注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。

2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。

```
typedef struct
{
    char from[CHAT_NAME_LEN];
    char to[CHAT_NAME_LEN];
    char text[CHAT_TEXT_LEN];
} ChatSendRequest;
```

图 41 聊天请求结构体

服务器写回客户端的包统一使用 ChatPacket：其中 CHAT_PACKET_REPLY 表示服务器确认或错误，CHAT_PACKET_MESSAGE 表示其他用户发来的聊天消息。

type 用来区分数据包类型：

- (1) **CHAT_PACKET_REPLY** 表示服务器对注册、登录或发送消息操作的确认或错误提示，此时 ok 为 1 表示成功，为 0 表示失败，message 保存提示内容
- (2) **CHAT_PACKET_MESSAGE** 表示其他用户发来的聊天消息，此时 from 保存发送者用户名，message 保存正文。客户端收到 ChatPacket 后只需要根据 type 选择不同打印格式

```
typedef enum
{
    CHAT_PACKET_REPLY = 1,
    CHAT_PACKET_MESSAGE = 2
} ChatPacketType;

typedef struct
{
    int type;
    int ok;
    char from[CHAT_NAME_LEN];
    char message[CHAT_TEXT_LEN];
} ChatPacket;
```

图 42 服务器写回客户端的包

服务器内部还定义了用户表结构体 User：不需要被客户端看到，只在服务器内维护。username 和 password 用于身份校验，fifo 用于记录用户当前客户端的私有 FIFO，online 用于判断能否接收消息。服务器用 users[CHAT_MAX_USERS] 保存最多 64 个用户，并用 user_count 记录当前已注册用户数量。服务器还使用 read_fds[3] 保存 3 个公共 FIFO 的读端，用 hold_fds[3] 保存对应写端；保留写端可以避免公共 FIFO 在没有客户端写入时反复读到 EOF，从而让 select() 监听更稳定

注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。

2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。

```
typedef struct
{
    char username[CHAT_NAME_LEN];
    char password[CHAT_PASSWORD_LEN];
    char fifo[CHAT_FIFO_PATH_LEN];
    int online;
} User;
```

图 43 User 结构体

5.3 client 实现思路

客户端程序的核心职责本质还是：创建自己的私有 FIFO，向服务器公共 FIFO 发送请求，同时监听键盘输入和服务器写回的消息。

- (1) **第一步初始化身份和目录**：main() 要求命令行参数为 **username password**，先调用 **valid_username()** 限制用户名只能包含字母、数字、下划线和短横线，避免用户名中包含 / 等字符导致私有 FIFO 路径被错误拼接，随后调用 **ensure_tree()** 自动创建实验要求的目录树，提高代码的鲁棒性

```
static int ensure_tree(void)
{
    return ensure_dir(CHAT_USER_HOME) == 0 &&
           ensure_dir(CHAT_APP_DIR) == 0 &&
           ensure_dir(CHAT_DATA_DIR) == 0 &&
           ensure_dir(CHAT_SERVER_FIFO_DIR) == 0 &&
           ensure_dir(CHAT_CLIENT_FIFO_DIR) == 0;
}

static int valid_username(const char *name)
{
    size_t i;

    if (name[0] == '\0')
        return 0;

    for (i = 0; name[i] != '\0'; i++)
    {
        if (!isalnum((unsigned char)name[i]) && name[i] != '_' && name[i] != '-')
            return 0;
    }

    return 1;
}

int main()
{
    if (argc != 3)
    {
        fprintf(stderr, "Usage: %s username password\n", argv[0]);
        return EXIT_FAILURE;
    }
    if (!valid_username(argv[1]))
    {
        fprintf(stderr, "Username may contain only letters, digits, '_' and '-' \n");
        return EXIT_FAILURE;
    }
}
```

图 44 初始化身份和目录代码

- 注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。
2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。

(2) **创建并打开客户端私有 FIFO**：生成当前客户端的私有 FIFO 路径，随后客户端以

O_RDONLY | O_NONBLOCK 打开读端，保存到 fifo_fd，用于读取服务器回复；同时再以 O_WRONLY | O_NONBLOCK 打开一次写端，保存到 hold_fd。

保留 hold_fd 的作用是让客户端自己的 FIFO 始终存在一个写端，避免服务器暂时没有写入者时客户端读端出现 EOF 并影响监听流程。退出时 cleanup() 会关闭两个文件描述符并 unlink(myfifo) 删除私有 FIFO

```
static int prepare_client_fifo(void)
{
    struct stat st;

    snprintf(myfifo, sizeof(myfifo), "%s/%s", CHAT_CLIENT_FIFO_DIR, username);
    if (stat(myfifo, &st) == 0)
    {
        if (!S_ISFIFO(st.st_mode))
        {
            fprintf(stderr, "%s exists but is not a FIFO\n", myfifo);
            return -1;
        }
    }
    else if (mkfifo(myfifo, 0600) == -1)
    {
        perror(myfifo);
        return -1;
    }

    fifo_fd = open(myfifo, O_RDONLY | O_NONBLOCK);
    if (fifo_fd == -1)
    {
        perror(myfifo);
        return -1;
    }

    hold_fd = open(myfifo, O_WRONLY | O_NONBLOCK);
    if (hold_fd == -1)
    {
        perror(myfifo);
        return -1;
    }

    return 0;
}
```

图 45 创建并打开客户端私有 FIFO 代码

(3) **根据用户命令构造请求**：客户端支持 4 个交互命令，/register, /login, /send <user> <text>, /quit;

注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。

2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。

```

static int handle_line(char *line)
{
    char *p; /* cursor */
    char *to; /* recipient */
    char *text; /* message text */

    line[strcspn(line, "\n")] = '\0'; /* trim newline */

    if (line[0] == '\0')
        return 0; /* ignore empty */

    if (strcmp(line, "/help") == 0)
    {
        print_help(); /* show commands */
        return 0;
    }

    if (strcmp(line, "/register") == 0)
    {
        send_auth_request(CHAT_FIFO_REGISTER); /* send register */
        return 0;
    }

    if (strcmp(line, "/login") == 0)
    {
        send_auth_request(CHAT_FIFO_LOGIN); /* send login */
        return 0;
    }

    if (strcmp(line, "/quit") == 0)
        return 1; /* request exit */
}

```

图 46 用户命令构造请求代码

- a) /register 和 /login 都调用 **send_auth_request()**，只是写入的公共 FIFO 不同：注册写入 CHAT_FIFO_REGISTER，登录写入 CHAT_FIFO_LOGIN。**send_auth_request()** 会把当前 username、password、myfifo 填入 ChatAuthRequest

```

static int send_auth_request(const char *server_fifo)
{
    ChatAuthRequest req; /* auth request */

    memset(&req, 0, sizeof(req)); /* clear struct */
    copy_string(req.username, sizeof(req.username), username); /* set username */
    copy_string(req.password, sizeof(req.password), password); /* set password */
    copy_string(req.fifo, sizeof(req.fifo), myfifo); /* set client FIFO */

    return write_request(server_fifo, &req, sizeof(req)); /* send to server */
}

```

图 47 /register 和 /login 请求代码

- 注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。
2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。

- b) **/send** 由 `handle_line()` 解析出目标用户名和消息正文，再调用 `send_chat_request()` 生成 `ChatSendRequest` 并写入 `CHAT_FIFO_MESSAGE`。底层统一由 `write_request()` 打开指定公共 FIFO，并把整个结构体一次写入

```
static int send_chat_request(const char *to, const char *text)
{
    ChatSendRequest req; /* chat send request */

    memset(&req, 0, sizeof(req)); /* clear struct */
    copy_string(req.from, sizeof(req.from), username); /* sender */
    copy_string(req.to, sizeof(req.to), to); /* recipient */
    copy_string(req.text, sizeof(req.text), text); /* message */

    return write_request(CHAT_FIFO_MESSAGE, &req, sizeof(req)); /* send to server */
}
```

图 48 根据用户命令构造请求代码

- (4) **select() 同时等待键盘和服务回复**：客户端主循环把 `STDIN_FILENO` 和 `fifo_fd` 都加入读集合，如果键盘可读，说明用户输入了命令，程序读取一行后交给 `handle_line()` 解析；如果私有 FIFO 可读，说明服务器写回了 `ChatPacket`，程序调用 `drain_packets()` 循环读取所有已到达的数据包。`print_packet()` 根据 `packet->type` 判断显示格式：服务器回复打印为 `[server] OK/ERROR: ...`，聊天消息打印为 `[发送者] 消息内容`。

因此客户端既能主动输入命令，也能异步收到其他用户发来的消息

```
while (1)
{
    FD_ZERO(&rfd); /* clear set */
    FD_SET(STDIN_FILENO, &rfd); /* watch stdin */
    FD_SET(fifo_fd, &rfd); /* watch FIFO */
    maxfd = fifo_fd > STDIN_FILENO ? fifo_fd : STDIN_FILENO; /* max fd */

    if (select(maxfd + 1, &rfd, NULL, NULL, NULL) == -1)
    {
        if (errno == EINTR)
            continue; /* retry on signal */
        perror("select");
        cleanup(EXIT_FAILURE); /* fatal error */
    }

    if (FD_ISSET(fifo_fd, &rfd))
    {
        drain_packets(); /* handle server packets */
        print_prompt(); /* restore prompt */
    }

    if (FD_ISSET(STDIN_FILENO, &rfd))
    {
        if (fgets(line, sizeof(line), stdin) == NULL)
            break; /* EOF */
        if (handle_line(line))
            break; /* /quit */
        print_prompt(); /* next prompt */
    }
}
```

图 49 客户端主循环代码

- 注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。
2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。

5.4 server 实现思路

服务器程序的核心职责是：创建 3 个公共 FIFO，维护用户表，用 select() 监听注册、登录和聊天请求，并根据请求类型完成校验和转发。

- (1) **创建目录和公共 FIFO**：服务器启动后调用 ensure_tree() 确保实验目录存在，然后分别调用三次 open_server_fifo()，其内部先用 ensure_fifo() 检查路径是否已经是 FIFO，不存在时用 mkfifo(path, 0666) 创建。之后服务器以 O_RDONLY | O_NONBLOCK 打开读端保存到 read_fds[index]，再以 O_WRONLY | O_NONBLOCK 打开写端保存到 hold_fds[index]。这样服务器可以长期监听公共 FIFO，不会因为暂时没有客户端写入而读到 EOF 后退出。

```
static int open_server_fifo(const char *path, int index)
{
    if (ensure_fifo(path) == -1)
        return -1;

    read_fds[index] = open(path, O_RDONLY | O_NONBLOCK);
    if (read_fds[index] == -1)
    {
        perror(path);
        return -1;
    }

    hold_fds[index] = open(path, O_WRONLY | O_NONBLOCK);
    if (hold_fds[index] == -1)
    {
        perror(path);
        return -1;
    }

    return 0;
}

if (!ensure_tree())
    exit(EXIT_FAILURE);

if (open_server_fifo(CHAT_FIFO_REGISTER, 0) == -1 ||
    open_server_fifo(CHAT_FIFO_LOGIN, 1) == -1 ||
    open_server_fifo(CHAT_FIFO_MESSAGE, 2) == -1)
    cleanup(EXIT_FAILURE);

printf("chatserver is ready\n");
printf("register fifo: %s\n", CHAT_FIFO_REGISTER);
printf("login fifo: %s\n", CHAT_FIFO_LOGIN);
printf("message fifo: %s\n", CHAT_FIFO_MESSAGE);
```

图 50 创建目录和公共 FIFO

- (2) **事件分派**：服务器主循环把 3 个公共 FIFO 的读端全部加入 select()

- a) 当 read_fds[0] 可读时调用 drain_auth_fifo(read_fds[0], 1) 处理注册，当 read_fds[1] 可读时调用 drain_auth_fifo(read_fds[1], 0) 处理登录

注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。
2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。

```

static void drain_auth_fifo(int fd, int is_register)
{
    ChatAuthRequest req;
    ssize_t nread;

    while (1)
    {
        nread = read(fd, &req, sizeof(req));
        if (nread == -1)
        {
            if (errno == EAGAIN || errno == EWOULDBLOCK)
                return;
            if (errno == EINTR)
                continue;
            perror("read");
            return;
        }

        if (nread == 0)
            return;

        if (nread != sizeof(req))
        {
            fprintf(stderr, "Ignoring incomplete auth request: %zd bytes\n", nread);
            continue;
        }

        if (is_register)
            handle_register(&req);
        else
            handle_login(&req);
    }
}

```

图 51 注册和登陆分派代码

b) 当 read_fds[2] 可读时调用 drain_chat_fifo(read_fds[2]) 处理聊天消息。

drain_auth_fifo() 和 **drain_chat_fifo()** 都使用循环读取，直到非阻塞读返回 EAGAIN 或 EWOULDBLOCK，这样可以一次处理完当前 FIFO 中积压的多个请求。

- 注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。
 2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。

```

static void drain_chat_fifo(int fd)
{
    ChatSendRequest req;
    ssize_t nread;

    while (1)
    {
        nread = read(fd, &req, sizeof(req));
        if (nread == -1)
        {
            if (errno == EAGAIN || errno == EWOULDBLOCK)
                return;
            if (errno == EINTR)
                continue;
            perror("read");
            return;
        }

        if (nread == 0)
            return;

        if (nread != sizeof(req))
        {
            fprintf(stderr, "Ignoring incomplete chat request: %zd bytes\n", nread);
            continue;
        }

        handle_chat(&req);
    }
}

```

图 52 send 分派代码

```

while (1)
{
    FD_ZERO(&rfd);
    FD_SET(read_fds[0], &rfd);
    FD_SET(read_fds[1], &rfd);
    FD_SET(read_fds[2], &rfd);

    maxfd = read_fds[0];
    if (read_fds[1] > maxfd)
        maxfd = read_fds[1];
    if (read_fds[2] > maxfd)
        maxfd = read_fds[2];

    if (select(maxfd + 1, &rfd, NULL, NULL, NULL) == -1)
    {
        if (errno == EINTR)
            continue;
        perror("select");
        cleanup(EXIT_FAILURE);
    }

    if (FD_ISSET(read_fds[0], &rfd))
        drain_auth_fifo(read_fds[0], 1);
    if (FD_ISSET(read_fds[1], &rfd))
        drain_auth_fifo(read_fds[1], 0);
    if (FD_ISSET(read_fds[2], &rfd))
        drain_chat_fifo(read_fds[2]);
}

```

图 53 主循环事件分派代码

- 注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。
- 2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。

- (3) **注册处理**: `handle_register()` 先检查用户名和密码是否为空, 再调用 `find_user()` 查找用户名是否存在。如果用户不存在且用户表未满, 就把用户名、密码、客户端私有 FIFO 保存到 `users[user_count]`, 将 `online` 初始化为 0, 再递增 `user_count`。
- 注册成功后, 服务器调用 `reply_to_fifo(req->fifo, 1, "register ok")` 给客户端返回成功信息; 如果用户名重复, 则返回 `username already exists`;

```
static void handle_register(const ChatAuthRequest *req)
{
    if (req->username[0] == '\0' || req->password[0] == '\0')
    {
        reply_to_fifo(req->fifo, 0, "username and password must not be empty");
        return;
    }

    if (find_user(req->username) != -1)
    {
        reply_to_fifo(req->fifo, 0, "username already exists");
        return;
    }

    if (user_count >= CHAT_MAX_USERS)
    {
        reply_to_fifo(req->fifo, 0, "server user table is full");
        return;
    }

    copy_string(users[user_count].username, sizeof(users[user_count].username), req->username);
    copy_string(users[user_count].password, sizeof(users[user_count].password), req->password);
    copy_string(users[user_count].fifo, sizeof(users[user_count].fifo), req->fifo);
    users[user_count].online = 0;
    user_count++;

    printf("registered user: %s\n", req->username);
    reply_to_fifo(req->fifo, 1, "register ok");
}
```

图 54 注册处理代码

- (4) **登录处理**: `handle_login()` 先通过 `find_user(req->username)` 查找用户。如果用户不存在, 返回 `username does not exist`; 如果密码不匹配, 返回 `password is incorrect`; 如果校验通过, 则把用户表中的 `fifo` 更新为本次登录客户端的私有 FIFO, 并把 `online` 设置为 1。这样同一个用户重新运行客户端后, 服务器仍能把消息发到最新的 FIFO

- 注: 1、报告内的项目或内容设置, 可根据实际情况加以调整和补充。
2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。

```

static void handle_login(const ChatAuthRequest *req)
{
    int index = find_user(req->username);

    if (index == -1)
    {
        reply_to_fifo(req->fifo, 0, "username does not exist");
        return;
    }

    if (strcmp(users[index].password, req->password) != 0)
    {
        reply_to_fifo(req->fifo, 0, "password is incorrect");
        return;
    }

    copy_string(users[index].fifo, sizeof(users[index].fifo), req->fifo);
    users[index].online = 1;

    printf("login user: %s, fifo: %s\n", req->username, req->fifo);
    reply_to_fifo(req->fifo, 1, "login ok");
}

```

图 55 登录处理代码

- (5) **聊天转发**: handle_chat() 先查找发送者和接收者，如果发送者不存在或未在线，服务器直接忽略该非法请求；如果接收者不存在，向发送者返回 target user does not exist；如果接收者存在但未在线，返回 target user is not online。当双方都在线时，服务器用 make_packet(&packet, CHAT_PACKET_MESSAGE, 1, req->from, req->text) 构造聊天数据包，并调用 send_packet(users[to_index].fifo, &packet) 写入接收者私有 FIFO。写入成功后，服务器再向发送者返回 message sent to <user>，表示这条消息已被服务器转发。

- 注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。
 2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。

```

static void handle_chat(const ChatSendRequest *req)
{
    int from_index = find_user(req->from);
    int to_index = find_user(req->to);
    ChatPacket packet;
    char message[CHAT_TEXT_LEN];

    if (from_index == -1 || !users[from_index].online)
        return;

    if (to_index == -1)
    {
        reply_to_fifo(users[from_index].fifo, 0, "target user does not exist");
        return;
    }

    if (!users[to_index].online)
    {
        reply_to_fifo(users[from_index].fifo, 0, "target user is not online");
        return;
    }

    make_packet(&packet, CHAT_PACKET_MESSAGE, 1, req->from, req->text);
    if (send_packet(users[to_index].fifo, &packet) == -1)
    {
        users[to_index].online = 0;
        reply_to_fifo(users[from_index].fifo, 0, "target fifo is not available");
        return;
    }

    snprintf(message, sizeof(message), "message sent to %s", req->to);
    reply_to_fifo(users[from_index].fifo, 1, message);
    printf("%s -> %s: %s\n", req->from, req->to, req->text);
}

```

图 56 聊天转发代码

- (6) **服务器退出**: 由 cleanup() 关闭所有公共 FIFO 的读写文件描述符, 并删除 FIFO_1、FIFO_2、FIFO_3。因此再次运行服务器时可以重新创建干净的公共 FIFO

- 注: 1、报告内的项目或内容设置, 可根据实际情况加以调整和补充。
2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。

```

static void cleanup(int status)
{
    int i;

    for (i = 0; i < 3; i++)
    {
        if (read_fds[i] != -1)
            close(read_fds[i]);
        if (hold_fds[i] != -1)
            close(hold_fds[i]);
    }

    unlink(CHAT_FIFO_REGISTER);
    unlink(CHAT_FIFO_LOGIN);
    unlink(CHAT_FIFO_MESSAGE);
    exit(status);
}

static void handle_signal(int sig)
{
    (void)sig;
    cleanup(EXIT_SUCCESS);
}

```

图 57 服务器退出代码

5.5 编译与测试

编译:

```
gcc -g -Wall -Wextra -o chatserver chatserver.c
```

```
gcc -g -Wall -Wextra -o chatclient chatclient.c
```

服务器终端: **./chatserver** 运行结果:

```

[szu@euler-container-30001 chatapplication]$ ./chatserver
chatserver is ready
register fifo: /home/szu/liwenjun2023150001/chatapplication/data/server_fifo/FIFO_1
login fifo:   /home/szu/liwenjun2023150001/chatapplication/data/server_fifo/FIFO_2
message fifo: /home/szu/liwenjun2023150001/chatapplication/data/server_fifo/FIFO_3

```

图 58 服务器初始运行

客户端 Alice:

```
./chatclient alice 123456
```

```
/register
```

```
/login
```

- 注: 1、报告内的项目或内容设置, 可根据实际情况加以调整和补充。
2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。

```

❖ chat> /send saul ^C[szu@euler-container-30001 chatapplication]$ ./chatclient alice 123456
client fifo: /home/szu/liwenjun2023150001/chatapplication/data/client_fifo/alice
commands:
  /register      register current username and password
  /login        login current username and password
  /send <user> <text> send text to another online user
  /quit         exit client
chat> /register
chat>
[server] OK: register ok
chat> /login
chat>
[server] OK: login ok
chat> █

```

图 59 客户端 Alice 注册和登陆

客户端 Bob:

`./chatclient bob 456789`

/register

/login

```

❖ chat> ^C[szu@euler-container-30001 chatapplication]$ ./chatclient bob 456789
client fifo: /home/szu/liwenjun2023150001/chatapplication/data/client_fifo/bob
commands:
  /register      register current username and password
  /login        login current username and password
  /send <user> <text> send text to another online user
  /quit         exit client
chat> /register
chat>
[server] ERROR: username already exists
chat> /login
chat>
[server] OK: login ok
chat> █

```

图 60 客户端 Bob 注册和登陆

Alice 发送消息: /send bob hello bob, this is alice

(1) Alice 输出:

```

chat> /send bob hello bob, this is alice
chat>
[server] OK: message sent to bob
chat> █

```

图 61 客户端 Alice 发送结果

(2) Bob 输出:

```

chat>
[server] OK: login ok
chat>
[alice] hello bob, this is alice
chat> █

```

图 62 客户端 Bob 发送结果

- 注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。
 2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。

(3) 服务器输出:

```
registered user: alice
login user: alice, fifo: /home/szu/liwenjun2023150001/chatapplication/data/client_fifo/alice
registered user: bob
login user: bob, fifo: /home/szu/liwenjun2023150001/chatapplication/data/client_fifo/bob
alice -> bob: hello bob, this is alice
█
```

图 63 服务器发送结果

5.6 异常情况分析

重复注册时，服务器查找到同名用户并返回：

```
chat> /register
chat>
[server] ERROR: username already exists
chat> █
```

图 64 重复注册结果

密码错误时，服务器比较保存的密码与登录请求密码并返回：

```
[szu@euler-container-30001 chatapplication]$ ./chatclient bob abcdefds
client fifo: /home/szu/liwenjun2023150001/chatapplication/data/client_fifo/bob
commands:
  /register      register current username and password
  /login         login current username and password
  /send <user> <text> send text to another online user
  /quit         exit client
chat> /login
chat>
[server] ERROR: password is incorrect
chat> █
```

图 65 密码错误结果

向不存在用户发送消息时，服务器无法找到接收方并返回：

```
chat> /send wenjun hello
chat>
[server] ERROR: target user does not exist
chat> █
```

图 66 不存在用户发送消息结果

向未在线用户发送消息时，服务器找到接收方但其 online 字段为 0，并返回：

```
chat> /send bob hello
chat>
[server] ERROR: target fifo is not available
chat> █
```

图 67 服务器发送结果

五、实验总结与体会

本次实验首先通过 GDB 调试原始程序，体会到系统编程中路径、头文件、信号和异常处理等细节会直接影响程序能否稳定运行。GDB 不只是用来查看崩溃位置，也可以用来观察宏、全局变量和运行时生成的字符串，从而定位程序逻辑是否符合实验要求。

注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。

2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。

FIFO 计算客户端-服务器实验说明了公共 FIFO 和私有 FIFO 的职责划分：公共 FIFO 适合接收请求，私有 FIFO 适合定向返回结果。如果所有客户端共用一个返回 FIFO，多个客户端同时运行时会出现响应归属不清的问题，因此使用 PID 生成客户端私有 FIFO 是更合理的设计。

信号实验展示了异步事件处理和普通函数调用的区别。SIGINT 默认会终止进程，但注册处理函数后可以转换为受控退出；SIGCHLD 则说明标准信号不排队，如果处理函数只回收一个子进程，就可能因为信号合并而遗漏子进程。因此处理子进程退出时应尽量循环调用 `wait()` 或 `waitpid()`，避免僵尸进程和状态计数不一致。

多用户聊天系统进一步体现了服务器在并发通信中的调度作用。服务器通过 `select()` 同时监听注册、登录和聊天三个 FIFO，用用户表保存用户名、密码、私有 FIFO 和在线状态，从而完成身份校验、消息路由和错误响应。虽然该系统仍是简单的单机单服务器模型，但已经包含客户端-服务器通信的核心结构。

- 注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。
2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。

[illegible]

成绩评定：

禹洪

指导教师签字：冯

2026 年 月

日

指导教师签字：冯

2026 年 月

2026 年 月

禹洪
2026 年 月
日

备注:	
-----	--

注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。
2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。

2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。

注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。
2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。